Highlights

TARA: Complexity-Adaptive Retrieval Refinement through Tool-Augmented ReAct Agents

Ruo Lee

- TARA: ReAct agent with six tools for autonomous retrieval refinement
- Statistically significant F1 +0.089 on 2WikiMultiHopQA ($p < 0.001$, $n = 200$)
- Cross-model validation reveals model–pipeline interaction effect (refusal asymmetry)
- DSPy typed signatures boost F1 by 0.075–0.190 over manual prompt engineering
- Comprehensive ablation with bootstrap significance testing across four datasets

TARA: Complexity-Adaptive Retrieval Refinement through Tool-Augmented ReAct Agents

Ruo Lee[a]

[a]*Department of Artificial Intelligence, Seoul School of Integrated Sciences & Technologies (aSSIST), Seoul, South Korea*

Abstract

Retrieval-Augmented Generation (RAG) has emerged as a practical approach to ground large language model (LLM) outputs in external knowledge. However, existing self-corrective RAG methods rely on fixed iterative loops with predetermined refinement strategies, limiting their ability to adapt to varying question complexity. We propose TARA, a framework that replaces fixed-loop refinement with a ReAct-based agent equipped with six specialized tools—passage search, query decomposition, quality evaluation, passage inspection, document structure browsing, and terminology mapping. The agent autonomously reasons about retrieval quality and selects appropriate tools in a Thought–Action–Observation loop, enabling adaptive refinement depth proportional to question complexity. The entire pipeline is implemented as a DSPy declarative program, enabling automatic prompt optimization through BootstrapFewShot and MIPROv2. Experiments on four datasets ($n = 200$; HotpotQA, 2WikiMultiHopQA, MuSiQue, and FinanceBench) with bootstrap significance testing demonstrated that TARA achieved statistically significant F1 improvement of +0.089 on 2WikiMultiHopQA ($p < 0.001$), with consistent positive trends on MuSiQue (+0.039). Cross-model validation with gpt-5-mini revealed a model–pipeline interaction effect where reasoning-oriented models exhibit refusal asymmetry in agentic pipelines. This finding provides evidence for model-aware pipeline selection. DSPy's typed signatures alone improved F1 by 0.075–0.190

Email address: comsa333@gmail.com (Ruo Lee)

on three of four datasets over manual prompt engineering, with automatic optimization yielding further gains. Ablation studies confirmed that the evaluation tool functions as a quality gate mechanism, structure-aware tools provide dataset-dependent benefits, and the agent converges on structured retrieval protocols.
Keywords: retrieval-augmented generation, agentic refinement, tool-augmented language model, DSPy, multi-hop question answering, self-corrective retrieval

---

1. Introduction

Large language models (LLMs) [1, 2, 3, 4] have demonstrated remarkable capabilities in natural language understanding and generation, yet they remain prone to hallucination and factual inaccuracy when responding from parametric knowledge alone [5]. Retrieval-Augmented Generation (RAG) addresses this limitation by grounding LLM outputs in externally retrieved evidence, establishing itself as a standard approach for knowledge-intensive tasks [6, 7, 8].

However, standard RAG assumes that a single retrieval pass produces sufficient evidence—an assumption that frequently fails for multi-hop questions requiring reasoning across multiple documents. When the initial retrieval misses a critical piece of evidence (*e.g.*, one of the two bridging facts in a 2-hop question), the generated answer is inevitably incomplete or incorrect. Self-corrective RAG methods [9, 10] address this by introducing iterative retrieval refinement: evaluating retrieved passages and re-retrieving when quality is insufficient.

Existing self-corrective approaches suffer from two fundamental limitations. First, they rely on *fixed iterative loops* with predetermined refinement strategies—retrieve, evaluate, refine query, repeat—that cannot adapt their strategy based on intermediate findings. A complex 4-hop question receives the same number of refinement iterations as a simple 2-hop question, regardless of whether the evidence gaps have been resolved. Second, they employ *manual prompt engineering* for each pipeline stage, requiring careful hand-tuning of prompt templates that

are fragile and difficult to optimize systematically.

We propose TARA (**T**ool-**A**ugmented **R**etrieval **A**gent), a framework that addresses both limitations through two key design decisions. First, we replace the fixed refinement loop with a **ReAct-based agent** [11] equipped with six specialized tools—passage search, query decomposition, quality evaluation, passage inspection, document structure browsing, and terminology mapping. The agent autonomously reasons about retrieval quality and selects appropriate tools in a Thought–Action–Observation loop, enabling adaptive refinement depth proportional to question complexity. Second, we implement the entire pipeline as a **DSPy declarative program** [12], replacing hand-crafted prompts with typed Signatures that enable both structured prompting and automatic optimization through BootstrapFewShot and MIPROv2 [13].

We evaluate TARA on four datasets: three multi-hop QA benchmarks (HotpotQA [14], 2WikiMultiHopQA [15], MuSiQue [16]) and an enterprise financial QA benchmark (FinanceBench [17]). Using $n = 200$ samples per dataset with paired bootstrap significance testing, we demonstrate that the agentic approach achieved statistically significant improvement over all baselines on 2WikiMultiHopQA (F1 +0.089, $p < 0.001$), with consistent positive trends on MuSiQue (+0.039). Cross-model validation with gpt-5-mini revealed a model–pipeline interaction effect—*Reasoning Model Refusal Asymmetry*—where reasoning-oriented models apply excessive evidentiary standards in agentic pipelines, providing practical guidance for model-aware pipeline selection. DSPy's typed Signatures alone improved F1 by 0.071–0.164 over manual prompts on three of four datasets, with additional gains from automatic optimization.

The main contributions of this paper are:

1. We propose TARA, a framework that replaces fixed-loop refinement with a ReAct-based agent equipped with six specialized tools, enabling autonomous, complexity-adaptive retrieval refinement for multi-hop question answering.
2. We implement the entire pipeline as a DSPy declarative program and

demonstrate that typed Signatures alone provide the single largest improvement in our experiments (F1 +0.075–0.190), with automatic optimization yielding further dataset-dependent gains.

3. We provide rigorous empirical evaluation ($n = 200$ with bootstrap significance testing) across four datasets, five pipeline variants, and two model families, including ablation studies on tool configurations, identification of boundary conditions (retrieval space saturation) and model–pipeline interaction effects (Reasoning Model Refusal Asymmetry), and quantification of the latency–quality trade-off.

2. Related Work

2.1. Retrieval-Augmented Generation

Retrieval-Augmented Generation [6] grounds LLM outputs in externally retrieved evidence, combining a retriever with a generator. Early approaches use a single retrieval pass with dense retrievers such as DPR [18] and ColBERT [19], both building on contextualized BERT [20] representations, or sparse methods like BM25 [21]. Pre-training with retrieval objectives (REALM [22]) and retrieval-augmented generation architectures (Fusion-in-Decoder [23], Atlas [24]) further improve knowledge grounding. Recent work improves retrieval quality through query rewriting [25], hypothetical document embeddings (HyDE) [26], LLM-augmented dense retrieval [27, 28], hybrid retrieval combining dense and sparse methods via Reciprocal Rank Fusion (RRF) [29], and document chunking strategies tailored to specific domains [30, 31]. For a comprehensive survey of RAG methods, we refer readers to Gao et al. [7].

Our work builds on this foundation by employing hybrid FAISS [32] + BM25 retrieval with RRF fusion as the base retrieval mechanism, enhanced by agentic refinement that iteratively improves retrieval quality through tool-augmented reasoning.

2.2. Self-Corrective RAG

Self-corrective approaches introduce feedback loops that evaluate and refine retrieved passages.

Self-RAG [10] trains a single LM to generate special reflection tokens that trigger on-demand retrieval and self-assess output quality. While effective, it requires fine-tuning a custom model with reflection tokens, limiting flexibility.

CRAG [9] introduces a lightweight retrieval evaluator that classifies documents as correct, incorrect, or ambiguous, triggering corrective actions including web search fallback and knowledge refinement through strip decomposition. However, CRAG's single-pass correction with binary evaluation provides limited refinement granularity.

Adaptive-RAG [33] routes questions to different retrieval strategies based on estimated complexity, using a classifier trained to distinguish simple, moderate, and complex questions. This complexity-aware approach is complementary to our work; our findings similarly show complexity-dependent improvement patterns.

Multi-hop dense retrieval methods offer an alternative approach: MDR [34] performs iterative retrieval by encoding the question with previously retrieved passages, Baleen [35] uses condensed summarization between retrieval hops, and BeamDR [36] applies beam search in the dense representation space. Datasets such as IIRC [37] further test retrieval under incomplete information.

Other iterative retrieval approaches include **FLARE** [38], which triggers retrieval when the model's generation confidence drops below a threshold; **IR-CoT** [39], which interleaves retrieval with chain-of-thought reasoning steps for multi-hop questions; and **Self-Ask** [40], which decomposes questions into sub-questions answered sequentially. Verify-and-Edit [41] post-hoc verifies chain-of-thought reasoning by retrieving evidence for each reasoning step, while REPLUG [42] treats the LM as a black box and prepends retrieved documents to the input.

Our approach differs from these methods in three aspects: (1) we use an autonomous *agent* rather than fixed correction rules; (2) the agent has access to

Table 1: Comparison of self-corrective RAG approaches. TARA uniquely combines multi-tool agentic refinement with declarative pipeline optimization.

Method	**Iterative**	**Tool Use**	**Declarative**	**Multi-hop**	**Agent**
Naive RAG	–	–	–	–	–
Self-RAG [10]	Selective	–	–	✓	Reflection
CRAG [9]	Single-pass	Web search	–	–	Rules
FLARE [38]	Adaptive	–	–	–	Confidence
IRCoT [39]	Interleaved	–	–	✓	CoT
Adaptive-RAG [33]	Selective	–	–	✓	Router
TARA	**Adaptive**	**4+2 tools**	**DSPy**	✓	**ReAct**

multiple specialized tools beyond simple re-retrieval; and (3) the entire pipeline is implemented as a *declarative program* amenable to automatic optimization.

Table 1 summarizes the key differences.

2.3. Tool-Augmented Language Model Agents

Tool-augmented LLMs extend language model capabilities through external tool invocation. MRKL [43] proposes a modular neuro-symbolic architecture combining LLMs with external knowledge sources and discrete reasoning modules. Toolformer [44] trains models to autonomously decide when and how to call tools via self-supervised learning. ART [45] automatically generates multi-step reasoning programs that interleave reasoning with tool calls. ReAct [11] interleaves reasoning traces with actions in a Thought–Action–Observation loop, enabling models to plan, execute, and refine their approach dynamically. Related reasoning paradigms include Tree of Thoughts [46], which explores multiple reasoning paths via tree search, and RAP [47], which frames LM reasoning as planning with a world model. Reflexion [48] extends this with verbal reinforcement learning, where agents reflect on failures to improve subsequent attempts.

Broader agent frameworks have also emerged: HuggingGPT [49] orchestrates specialized AI models as tools, ToolLLM [50] facilitates tool mastery across

16,000+ APIs, and AgentBench [51] provides a benchmark for evaluating LLMs as autonomous agents across diverse environments. Additional benchmarks such as SWE-bench [52] and WebArena [53] evaluate agents in software engineering and web navigation contexts, respectively.

In the RAG context, agentic approaches have emerged that use LLM agents to orchestrate retrieval [54]. Search-o1 [55] integrates agentic search into reasoning chains of large reasoning models, dynamically retrieving knowledge when the model encounters uncertainty during extended reasoning. PlanRAG [56] introduces a plan-then-retrieve strategy where the LLM first generates an analysis plan before issuing retrieval queries, targeting decision-making over structured databases. Earlier work explores multi-step retrieval agents that decompose complex queries [57], perform iterative search, and synthesize evidence from multiple sources. However, most existing agentic RAG systems focus on a single retrieval tool with fixed orchestration logic.

TARA differs in providing a diverse tool set (four core, two domain-adaptive) with autonomous tool selection through ReAct reasoning, combined with a declarative DSPy implementation that enables automatic prompt optimization—a combination not explored in prior agentic RAG work.

2.4. Declarative Language Model Programming

The broader field of prompt optimization has explored various approaches to automatically improve LM prompts. AutoPrompt [58] uses gradient-guided search over discrete tokens, APE [59] treats prompt generation as a natural language synthesis problem, and OPRO [60] uses LLMs themselves as optimizers. More recently, TextGrad [61] backpropagates natural language feedback through computational graphs for end-to-end optimization.

DSPy [12], building on the Demonstrate-Search-Predict framework [62], introduces a programming model for LM-based systems based on declarative *Signatures* (typed input/output specifications) and composable *Modules* (*e.g.*, ChainOfThought, ReAct). This separates the program logic from the prompt

implementation, enabling automatic optimization through compilers like BootstrapFewShot and MIPROv2 [13].

BootstrapFewShot collects successful execution traces and uses them as few-shot demonstrations, automatically generating in-context examples. MIPROv2 extends this by jointly optimizing natural language instructions and demonstrations using Bayesian surrogate models.

While DSPy has been applied to various NLP tasks, its application to self-corrective RAG pipelines—particularly in combination with ReAct agents and multi-tool orchestration—has not been systematically studied. Our work demonstrates that DSPy's structured prompting provides substantial benefits even without optimization (+0.075–0.190 F1), with additional gains from automatic optimization that are dataset-dependent.

3. Method

This section presents TARA (**T**ool-**A**ugmented **R**etrieval **A**gent), our complexity-adaptive self-corrective RAG framework. We first formalize the problem (§3.1), then describe the overall architecture (§3.2), the ReAct agent and its tools (§3.3), the self-corrective refinement mechanism (§3.4), the DSPy declarative pipeline (§3.5), and the hybrid retrieval system (§3.6).

3.1. Problem Formulation

Given a question q and a document corpus $\mathcal{C} = \{d_1, d_2, \ldots, d_N\}$, the goal of retrieval-augmented generation is to produce an answer a by first retrieving a set of relevant passages $\mathcal{P} \subset \mathcal{C}$ and then generating a conditioned on both q and $\mathcal{P}$:

$$a = \text{Generate}(q, \mathcal{P}), \quad \mathcal{P} = \text{Retrieve}(q, \mathcal{C}). \tag{1}$$

In self-corrective RAG, the retrieval step is iteratively refined based on a quality assessment function $\text{Eval}(q, \mathcal{P}) \to s \in [0, 100]$ that scores the retrieved passages. Existing approaches [9] implement this as a fixed loop:

$$\mathcal{P}^{(t+1)} = \text{Retrieve}(q^{(t+1)}, \mathcal{C}), \quad q^{(t+1)} = \text{Refine}(q^{(t)}, s^{(t)}), \tag{2}$$

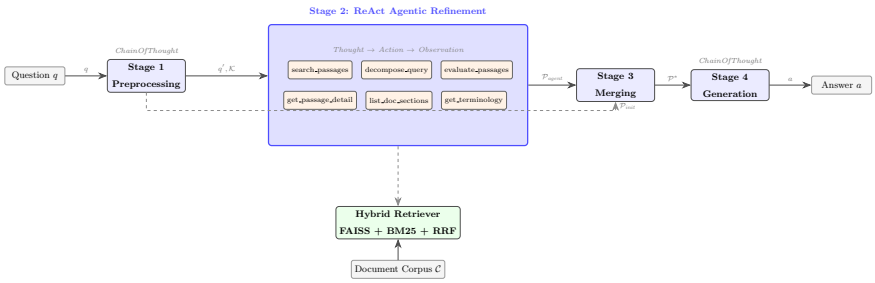

Figure 1: Overall architecture of TARA. The pipeline consists of four stages: (1) query preprocessing, (2) ReAct agentic refinement with six specialized tools, (3) passage merging, and (4) answer generation. Dashed arrows indicate the initial retrieval path and retriever connection.

iterating until $s^{(t)} \geq \tau$ (quality threshold) or $t \geq T$ (maximum iterations).

We replace this fixed loop with an *agentic* formulation. Let $\mathcal{T} = \{t_1, t_2, \ldots, t_K\}$ be a set of tools available to a ReAct agent [11]. At each step i, the agent produces a thought h_i, selects a tool $t_{k_i} \in \mathcal{T}$, observes the result o_i, and updates its internal state:

$$(h_i, t_{k_i}, o_i) = \text{ReAct}(q, \mathcal{H}_{<i}), \quad \mathcal{H}_{<i} = \{(h_j, t_{k_j}, o_j)\}_{j<i}. \tag{3}$$

The agent terminates when it determines that the accumulated passages are sufficient, producing the final passage set $\mathcal{P}^*$ and answer $a = \text{Generate}(q, \mathcal{P}^*)$.

3.2. Overall Architecture

Figure 1 illustrates the TARA pipeline, and Figure 2 contrasts it with prior approaches. The pipeline consists of four stages:

1. **Query Preprocessing.** The input question q is rephrased into a standalone search query q' and a set of search keywords $\mathcal{K}$ using a ChainOfThought module (DSPy `PreprocessSignature`). Optionally, a Hypothetical Document Embedding (HyDE) [26] query is generated for improved dense retrieval.
2. **ReAct Agentic Refinement.** The preprocessed query is passed to a ReAct agent equipped with six tools (§3.3). The agent iteratively searches,

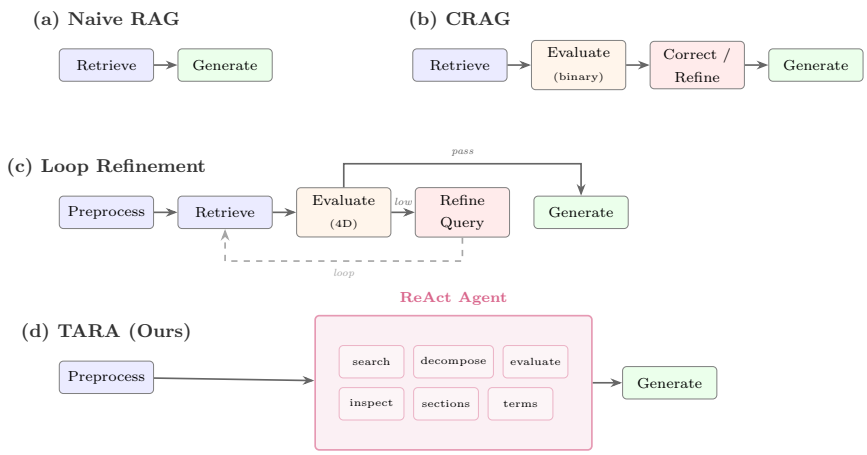

Figure 2: Comparison of pipeline architectures. (a) Naive RAG: single-pass retrieval. (b) CRAG: binary evaluation with single-pass correction. (c) Loop Refinement: fixed retrieve–evaluate–refine cycle. (d) TARA: ReAct agent with six tools for adaptive refinement.

decomposes, evaluates, and inspects passages through autonomous reasoning. This stage replaces the fixed iterative loop in prior work.

3. **Passage Merging.** Agent-selected passages are merged with initial retrieval results (from Stage 1) using priority-based deduplication: agent-refined passages take precedence, followed by initial results, capped at $M = 30$ passages.
4. **Answer Generation.** A ChainOfThought generator (DSPy `QnAGenerateSignature`) produces the final answer conditioned on the merged passages, with adaptive response formatting based on question type.

3.3. ReAct Agent and Tool Design

The core of TARA is a ReAct agent [11] that interleaves reasoning (Thought) with tool execution (Action) and result observation (Observation). Unlike fixed-loop refinement, the agent dynamically decides *which* tools to call, *how many* times, and *when* to stop based on its assessment of the current retrieval state.

The agent operates through a DSPy Signature (the `AgenticRefinementSignature`) that specifies typed inputs and outputs. The signature's docstring

serves as a structured protocol, guiding the agent through a recommended sequence of steps while allowing deviations based on the agent's reasoning.

3.3.1. Tool Set

The agent has access to six tools, each implemented as a closure over shared pipeline components (retriever, indexer, evaluator). The first four are *core tools* invoked on virtually every question; the remaining two are *domain-adaptive auxiliary tools* whose utility depends on corpus characteristics (see RQ4 in §6.2):

1. `search_passages`$(q, \mathcal{K})$: Executes hybrid retrieval (FAISS + BM25 with RRF fusion) and returns passage previews (first 200 characters) to conserve context. This is the primary retrieval tool, called on average 3–4 times per question.
2. `decompose_query`(q): Decomposes a complex multi-hop question into 2–4 independent sub-questions using a DSPy `DecomposeQuerySignature`. Returns a boolean `is_multi_hop` flag and the sub-question list.
3. `evaluate_passages`$(q, \mathcal{P})$: Assesses passage quality on four dimensions—Relevance (0–30), Coverage (0–25), Specificity (0–25), Sufficiency (0–20)—producing a total score $s \in [0, 100]$ with an action decision (`output`/`refine`) and refinement feedback. A mandatory fallback ensures this tool is called at least once per question.
4. `get_passage_detail`(id): Retrieves the full text of a passage by ID, enabling the agent to read complete content after identifying promising passages from search previews. This implements a progressive information disclosure strategy.
5. `list_document_sections`(id): Browses the hierarchical structure (sections, table of contents) of a source document, enabling structure-aware navigation for documents with rich organizational hierarchy.
6. `get_terminology`$(term)$: Maps user-facing terms to document-internal terminology, bridging vocabulary gaps between questions and corpus language.

An optional `calculate` tool is available for domain-specific numerical computations (*e.g.*, financial calculations) but is not part of the six core retrieval tools.

3.3.2. Agent Protocol

The agent's signature includes a structured protocol embedded in the docstring:

1. Call `decompose_query` to determine multi-hop structure.
2. Call `search_passages` with the initial query; if multi-hop, search each sub-question separately.
3. Optionally call `get_passage_detail` on promising results.
4. **Mandatory**: Call `evaluate_passages` on the accumulated passage set.
5. If score $< \tau$: use evaluation feedback to refine the query and search again.
6. Call `finish` when quality is sufficient or further search is unlikely to improve results.

The protocol is a recommendation, not a constraint—the agent may deviate based on its reasoning. However, a mandatory evaluate fallback ensures that quality assessment is never entirely skipped: if the agent reaches `finish` without calling `evaluate_passages`, the pipeline automatically runs the evaluation tool on the accumulated passages.

Figure 3 shows an example agent trajectory for a 3-hop question.

3.4. Self-Corrective Refinement Mechanism

The self-corrective loop in TARA differs from prior work in three key aspects:

Adaptive Depth. Rather than iterating a fixed number of times, the agent decides when to stop based on quality signals. Simple questions may require only one search–evaluate cycle, while complex multi-hop questions trigger multiple decompose–search–evaluate iterations. The maximum number of ReAct iterations is set to $I = 5$ to prevent unbounded execution.

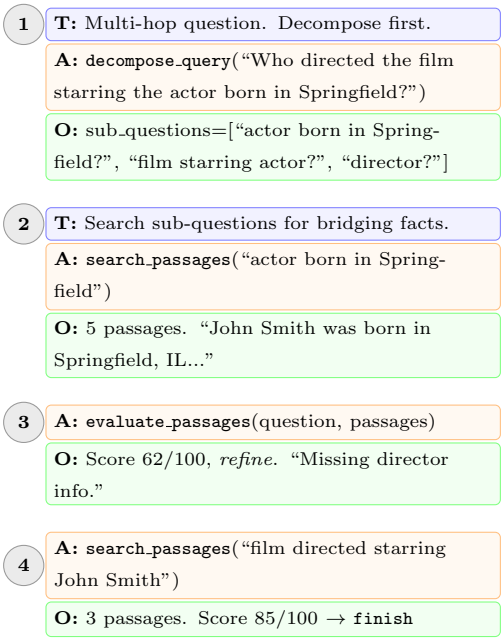

Figure 3: Example ReAct trajectory for a 3-hop question. T=Thought, A=Action, O=Observation. The agent decomposes, searches, evaluates, and refines based on feedback.

Progressive Leniency. The quality threshold decreases with each evaluation attempt to prevent over-iteration:

$$\tau_{\text{eff}}^{(t)} = \tau - t \cdot \delta, \quad \delta = 5, \tag{4}$$

where $\tau = 40$ is the base threshold. This ensures the agent eventually terminates with the best available passages rather than continuing to search for marginally improved results.

Structured Feedback. When the evaluation score falls below the threshold, the evaluation tool provides targeted feedback—suggested queries, keywords to add or remove—that directly informs the agent's next search action. This feedback loop replaces generic retry logic with informed refinement.

3.5. DSPy Declarative Pipeline

The entire TARA pipeline is implemented using DSPy [12], a framework for programming LM-based systems through declarative *Signatures* and com-

posable *Modules*.

3.5.1. Signature Design

Each pipeline stage is defined as a DSPy Signature with typed input and output fields:

- **PreprocessSignature**: (question, history) → (rephrased_question, keywords, topic)
- **EvaluationSignature**: (question, passages, retry_count) → (scores_{4D}, action, feedback)
- **QnAGenerateSignature**: (question, passages) → (answer, footnotes)
- **DecomposeQuerySignature**: (question) → (is_multi_hop, sub_questions)
- **AgenticRefinementSignature**: (question, query, keywords, τ) → (passages, action)

The typed field descriptions serve as structured prompts, replacing hand-crafted JSON templates. Each Signature's docstring provides task-specific instructions and scoring guidelines.

3.5.2. Module Composition

Signatures are composed with DSPy modules:

- `ChainOfThought` [63]: Wraps the PreprocessSignature and QnAGenerateSignature to elicit step-by-step reasoning before outputs.
- `Predict`: Wraps the EvaluationSignature for direct structured output.
- `ReAct`: Wraps the AgenticRefinementSignature with tool integration, implementing the Thought–Action–Observation loop.

3.5.3. Automatic Optimization

DSPy enables automatic prompt optimization without modifying the pipeline code:

- **BootstrapFewShot** [12]: Collects successful execution traces as few-shot demonstrations. Each trace includes the full input–output chain, providing in-context examples that improve LM performance.
- **MIPROv2** [13]: Generates and optimizes natural language instructions for each Signature using Bayesian surrogate models. It jointly optimizes instructions and few-shot examples across all pipeline modules.

Both optimizers use a training set of 50 examples with F1 score as the optimization metric. The optimization cost is amortized: training is performed once, and the optimized prompts (few-shot demonstrations and/or instructions) are reused for all subsequent queries.

3.6. Hybrid Retrieval

TARA uses a hybrid retrieval system combining dense and sparse search with Reciprocal Rank Fusion (RRF) [29]:

Dense Retrieval. Documents are embedded using a sentence transformer model and indexed with FAISS [32] for approximate nearest neighbor search. Given a query q, the dense retriever returns the top-k passages by cosine similarity.

Sparse Retrieval. BM25 [21] provides lexical matching for keyword-heavy queries, particularly effective for entity names and technical terms that may not be well-captured by dense embeddings.

Reciprocal Rank Fusion. The results from both retrievers are combined using RRF:

$$\mathrm{RRF}(d) = \sum_{r \in \{dense, sparse\}} \frac{1}{\kappa + \mathrm{rank}_r(d)}, \tag{5}$$

where $\kappa = 60$ is a smoothing constant and $\mathrm{rank}_r(d)$ is the rank of document d in retriever r's result list. The fused ranking is used as the final retrieval output, with the top $k = 50$ results returned per query.

Table 2: Dataset statistics. Multi-hop QA datasets include context paragraphs as self-contained retrieval corpora. FinanceBench provides evidence passages from SEC filings.

Dataset	Domain	Hops	Answer Type	Source
HotpotQA	Wikipedia	2	Extractive	[14]
2WikiMultiHopQA	Wikipedia	2–4	Extractive	[15]
MuSiQue	Wikipedia	2–4	Extractive	[16]
FinanceBench	SEC filings	1–3	Analytical	[17]

4. Experimental Setup

4.1. Datasets

We evaluate TARA on four datasets spanning multi-hop reasoning and enterprise document QA, summarized in Table 2. For a comprehensive survey of multi-hop QA methods, we refer readers to Mavi et al. [64].

HotpotQA [14] is a two-hop QA dataset over Wikipedia, where each question requires reasoning over two supporting paragraphs mixed with distractor paragraphs. We use the `distractor` setting from the validation split.

2WikiMultiHopQA [15] extends multi-hop reasoning to 2–4 hops, with questions constructed through systematic composition of single-hop questions from two Wikipedia articles. This dataset tests deeper reasoning chains than HotpotQA.

MuSiQue [16] (**Mu**lti-hop **Si**ngle-hop **Que**stion composition) provides 2–4 hop questions with controlled compositionality, designed to resist shortcut reasoning. It is generally considered the most challenging of the three multi-hop datasets.

FinanceBench [17] provides questions over real SEC filings (10-K, 10-Q, 8-K reports), requiring numerical reasoning and domain-specific terminology understanding. Related financial QA benchmarks include FinQA [65] for numerical reasoning over financial tables and DocFinQA [66] for long-context financial documents. This dataset evaluates practical applicability in enterprise

settings where external web search is unavailable.

Each dataset provides its own context paragraphs (supporting documents plus distractors), which we use as the retrieval corpus. We build separate FAISS and BM25 indices per dataset.

4.2. Baselines

We compare TARA against four baselines of increasing complexity, all sharing the same retrieval infrastructure and generation model for fair comparison:

1. **Naive RAG**: Single-pass retrieve-and-generate with no evaluation or correction. Uses the same hybrid retriever and ChainOfThought generator.
2. **CRAG Replica** [9]: Reproduces the core CRAG mechanism with binary relevance evaluation (correct/incorrect/ambiguous), knowledge refinement via strip decomposition, and internal search fallback. Two adaptations were made for fair comparison: (a) CRAG's web search fallback is replaced with internal corpus re-retrieval, as our evaluation setting assumes a closed corpus without external web access—a realistic constraint for enterprise deployments; (b) the original fine-tuned T5-large evaluator is replaced with LLM-based evaluation using the same model as other pipelines, ensuring the same compute class across all baselines. These adaptations disadvantage CRAG relative to its original design; we discuss this in §5.1.
3. **Single-Pass**: Preprocessing (query rephrasing + keyword extraction) followed by hybrid retrieval and generation. Includes one evaluation step but no iterative refinement.
4. **Loop Refinement**: Fixed for-loop self-corrective pipeline implementing iterative retrieval, 4D quality evaluation, and targeted query refinement. Iterates up to $T = 3$ times based on evaluation scores. This represents the non-agentic version of our refinement approach.

The key difference between Loop Refinement and TARA is the refinement strategy: Loop follows a predetermined retrieve–evaluate–refine sequence, while

TARA autonomously selects tools and decides when to stop through ReAct reasoning.

Baseline scope. We do not include Self-RAG [10] or Adaptive-RAG [33] as direct baselines because they require additional trained components (Self-RAG: a fine-tuned model with reflection tokens; Adaptive-RAG: a trained complexity classifier) that would confound the comparison of refinement strategies. All our baselines share the same retrieval infrastructure, generation model, and closed-corpus setting to isolate the effect of refinement mechanism design.

4.3. Evaluation Metrics

We use three primary metrics and one secondary metric:

Primary Metrics.

- **Exact Match (EM)**: Binary indicator of whether the normalized prediction exactly matches the reference answer, after lowercasing, removing articles/punctuation/whitespace, and stripping footnote markers.
- **Token-level F1**: Harmonic mean of token-level precision and recall between the normalized prediction and reference, following the standard SQuAD evaluation protocol. This awards partial credit for incomplete but overlapping answers.
- **LLM-as-Judge** [67]: A DSPy-based correctness judge that evaluates whether the prediction is semantically equivalent to the reference answer, using the same evaluation model. This captures cases where the answer is correct but lexically different from the reference (*e.g.*, "Dutch" vs. "the Netherlands").

Secondary Metric.

- **ROUGE-L** [68]: Longest common subsequence-based overlap score, providing an alternative view of answer quality. Reported in supplementary material.

We also report **average latency** per question to quantify the computational cost–quality trade-off. We note that alternative evaluation frameworks exist for RAG systems, including RAGAS [69] for component-level evaluation, FActScore [70] for fine-grained factual precision, and BERTScore [71] for semantic similarity. We focus on EM, F1, and LLM-as-Judge as they directly measure answer correctness and are standard in the multi-hop QA literature.

4.4. Implementation Details

Models. Primary experiments use Gemini Flash Lite [72][1] for all four LLM slots (preprocessing, evaluation, generation, agent) via LiteLLM. For cross-model validation (RQ1), we replicate experiments with gpt-5-mini[2], a reasoning-oriented model from a different architecture family, to assess generalizability. An independent LLM-as-Judge (gpt-4.1-nano) evaluates answer correctness by comparing predicted and reference answers via a DSPy-based correctness signature. The judge receives the question, reference answer, and predicted answer, and outputs a binary correctness decision with chain-of-thought reasoning. This model is independent of all pipeline models to avoid evaluation bias; it captures semantic equivalences that token-level F1 misses (*e.g.*, "Dutch" vs. "the Netherlands"). Embeddings use all-MiniLM-L6-v2 (384 dimensions) via sentence-transformers [73]. Temperature is set to 0.0 for reproducibility.

Retrieval. FAISS indices use inner product search with L2-normalized embeddings (equivalent to cosine similarity). BM25 uses the rank-bm25 library with default parameters. RRF fusion uses $\kappa = 60$. The retriever returns top-$k = 50$ results per query, with a maximum of $M = 30$ accumulated passages.

Agent. The ReAct agent operates with a maximum of $I = 5$ reasoning–action iterations. The quality threshold is $\tau = 40$ with progressive threshold relaxation $\delta = 5$ per evaluation attempt (minimum effective threshold: 20). The

[1] Model ID: `gemini/gemini-3.1-flash-lite-preview`
[2] Model ID: `gpt-5-mini`, `reasoning_effort=low`.

maximum retry count for the evaluation tool is $T = 3$. The maximum number of accumulated passages is $M = 30$.

DSPy Optimization. BootstrapFewShot and MIPROv2 optimizers use a training set of 50 examples per dataset, with 20 reserved for validation and the remainder for testing ($N = 130$ for multi-hop datasets, $N = 80$ for FinanceBench). Token-level F1 serves as the optimization metric. Optimized prompts include up to 4 few-shot demonstrations.

Infrastructure. Experiments run on a single machine with Python 3.11, DSPy 3.1+, and FAISS-CPU. All code, configurations, and experiment scripts are available at `https://github.com/comsa33/self-corrective-rag`.

Sample Size and Statistical Testing. Main experiments use $n = 200$ samples per dataset, with FinanceBench evaluated on the full dataset ($n = 150$). Statistical significance is assessed via paired bootstrap testing [74] with 10,000 resamples and Bonferroni correction for multiple comparisons. We report 95% bootstrap confidence intervals for all primary comparisons.

Reproducibility. All experiments use a fixed random seed (42) for dataset sampling. Configuration is managed through YAML files with per-experiment overrides. The entire pipeline—from dataset preparation through index building to experiment execution—is automated via CLI scripts.

5. Results and Analysis

This section presents the main experimental results. All experiments use $n = 200$ samples per dataset (FinanceBench: $n = 150$, full dataset) unless otherwise noted. We address three research questions: RQ1 compares TARA against baselines (§5.1), RQ2 analyzes the agent's tool usage patterns (§5.2), and RQ5 evaluates DSPy optimization effects (§5.3). Ablation studies for RQ3 (evaluation tool variants) and RQ4 (structure-aware tools) are presented in §6.

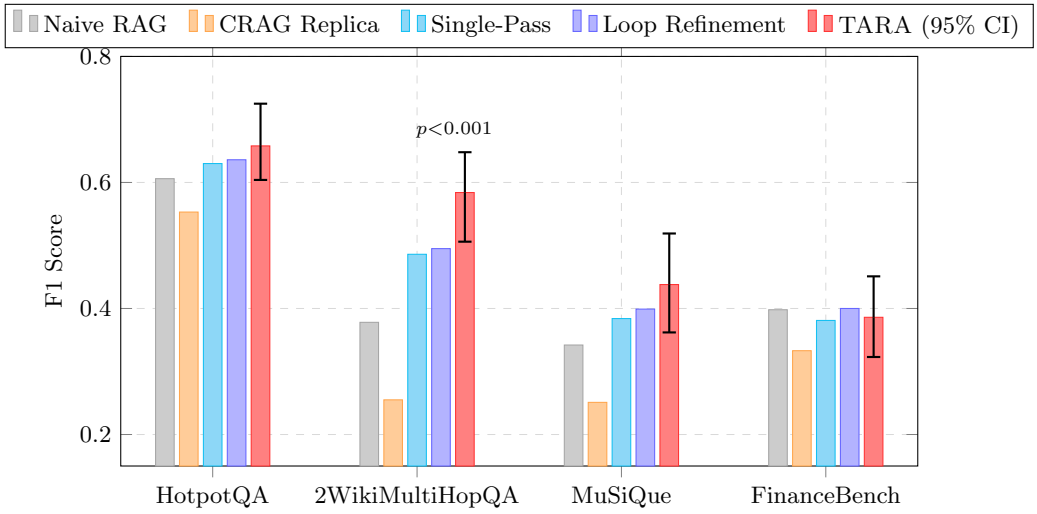

Figure 4: F1 comparison across four datasets (Gemini Flash Lite, $n = 200$). TARA achieves the highest F1 on three of four datasets, with statistically significant improvement on 2WikiMultiHopQA ($p < 0.001$). Error bars on TARA indicate 95% bootstrap confidence intervals (10,000 resamples; Table 4). On FinanceBench, all iterative methods perform comparably (retrieval space saturation).

5.1. RQ1: Agentic vs. Baseline Pipelines

Does ReAct-based agentic refinement improve answer quality compared to fixed-loop refinement and single-pass baselines?

Table 3 and Figure 4 present the main results across four datasets using Gemini Flash Lite.

Complex multi-hop questions. TARA achieves the best performance on both complex multi-hop datasets: 2WikiMultiHopQA (F1 0.584, +0.089 over Loop Refinement) and MuSiQue (F1 0.438, +0.039 over Loop). These improvements are consistent across all three metrics (EM, F1, Judge), indicating genuine quality gains rather than metric-specific artifacts. The LLM-as-Judge scores show the largest gaps on 2WikiMultiHopQA (+0.140), where the judge identifies correct multi-hop reasoning even when token overlap is moderate. As we show in §5.1.1, the 2WikiMultiHopQA improvement is statistically significant ($p < 0.001$) across all baseline comparisons.

Table 3: Main results (RQ1, Gemini Flash Lite). Best result per dataset in **bold**. $n = 200$ per dataset (FinanceBench: $n = 150$, full dataset). LLM-as-Judge uses gpt-4.1-nano as an independent evaluator.

	HotpotQA			**2WikiMultiHopQA**			**MuSiQue**			**FinanceBench**		
Pipeline	EM	F1	Judge	EM	F1	Judge	EM	F1	Judge	EM	F1	Judge
Naive RAG	.445	.606	.700	.275	.378	.400	.235	.342	.335	.167	.398	**.580**
CRAG Replica	.400	.553	.645	.130	.255	.320	.175	.251	.225	.107	.333	.487
Single-Pass	.470	.630	.720	.375	.486	.495	.280	.384	.360	.160	.381	.507
Loop Refinement	.480	.636	.730	.380	.495	.505	.300	.399	.380	**.180**	**.400**	.533
TARA	**.505**	**.658**	**.765**	**.470**	**.584**	**.645**	**.330**	**.438**	**.405**	.153	.386	.527
Δ *vs. Loop*	+.025	+.022	+.035	+.090	+.089	+.140	+.030	+.039	+.025	−.027	−.014	−.006

Simple 2-hop questions. On HotpotQA, TARA leads across all metrics (F1 0.658, EM 0.505, Judge 0.765), but the margin over Loop Refinement is small (F1 +0.022). This suggests that 2-hop questions—where the required evidence can typically be retrieved in one or two search passes—do not fully exercise the agent's adaptive refinement capabilities. The overhead of multi-step ReAct reasoning provides limited benefit when a single well-preprocessed retrieval pass is already sufficient.

Enterprise documents. On FinanceBench, Loop Refinement marginally outperforms TARA in F1 (0.400 vs. 0.386, $\Delta = -0.014$) and EM (0.180 vs. 0.153). Notably, Naive RAG achieves the highest Judge score (0.580), outperforming all iterative methods. This pattern—where iterative refinement does *not* improve and may slightly degrade answer quality—suggests a boundary condition for our approach. We analyze this effect in detail in §7.2, attributing it to retrieval space saturation in FinanceBench's small corpus (211 passages).

CRAG Replica underperformance. The CRAG Replica consistently ranks lowest, which is expected: our implementation replaces CRAG's web search fallback with internal re-retrieval (to match our enterprise-constrained setting) and uses LLM-based evaluation instead of a fine-tuned T5-large evaluator. These architectural constraints substantially disadvantage the CRAG approach.

Table 4: Statistical significance (F1, paired bootstrap test, 10,000 resamples, Bonferroni-corrected $\alpha = 0.0125$). Top row: TARA F1 with 95% bootstrap confidence interval. Lower rows: F1 difference Δ vs. each baseline (from Table 3); p-values from paired bootstrap. Significance: $^{***}p < 0.001$.

	HotpotQA	**2WikiMultiHopQA**	**MuSiQue**	**FinanceBench**
TARA F1 [95% CI]	.658 [.604, .725]	.584 [.506, .648]	.438 [.362, .519]	.386 [.323, .451]
Δ vs. Loop	+.022	+.089***	+.039	−.014
Δ vs. CRAG	+.105***	+.329***	+.187***	+.053
Δ vs. Single	+.028	+.098***	+.054	+.005
Δ vs. Naive	+.052	+.206***	+.096***	−.012

Key finding. The agentic advantage is *complexity-dependent*: improvements grow with question complexity (number of reasoning hops and retrieval difficulty), while simple questions and small-corpus settings show no benefit. This pattern is consistent with the hypothesis that autonomous tool selection provides value primarily when the retrieval task requires adaptive strategy.

5.1.1. Statistical Significance

To rigorously assess the reliability of observed differences, we perform paired bootstrap significance testing [74] with 10,000 resamples and Bonferroni correction for multiple comparisons. Table 4 reports 95% confidence intervals for TARA and pairwise comparisons against each baseline.

2WikiMultiHopQA: strongest evidence. TARA achieves statistically significant improvement over *all four baselines* on 2WikiMultiHopQA ($p < 0.001$ in each case), with the largest effect against CRAG ($\Delta = +0.329$). The 95% confidence interval $[0.506, 0.648]$ does not overlap with the confidence intervals of Loop $[0.433, 0.557]$ or CRAG $[0.209, 0.305]$, providing strong evidence that the improvement is not due to sampling variability. A hop-level breakdown reveals that the improvement is concentrated in 4-hop questions ($n = 50$): TARA achieves F1 0.628 vs. Loop's 0.323 ($\Delta = +0.305$), while on 2-hop questions ($n = 150$) the gap is modest (+0.018). This confirms that the agentic advan-

tage scales with reasoning complexity—the agent's ability to decompose, search sub-questions independently, and synthesize evidence provides the greatest benefit when multiple retrieval steps are required.

MuSiQue: significant vs. weaker baselines. Against CRAG (+0.187, $p < 0.001$) and Naive (+0.096, $p < 0.001$), the improvement is significant. Against Loop (+0.039) and Single-Pass (+0.054), the positive trend does not reach significance, indicating that the refinement mechanisms converge for 3–4 hop questions that overlap in difficulty.

HotpotQA: comparable performance. No pairwise comparison reaches significance except against CRAG (+0.105, $p < 0.001$), confirming that 2-hop questions do not discriminate between refinement approaches.

FinanceBench: no significant differences. All pairwise comparisons are non-significant, with confidence intervals substantially overlapping. The marginally negative delta against Loop (−0.014) is well within noise. This statistical confirmation of the null result is consistent with the retrieval space saturation hypothesis discussed in §7.2.

Effect sizes and statistical power. To assess practical significance beyond p-values, we report Cohen's d (paired) for each comparison. On 2WikiMultiHopQA, effect sizes range from small-to-medium ($d = 0.235$ vs. Loop and Single-Pass) to large ($d = 0.866$ vs. CRAG), indicating practically meaningful improvements. A sensitivity analysis (paired t-test, Bonferroni-corrected $\alpha = 0.0125$, 80% power) indicates that our $n = 200$ design can detect effects of $d \geq 0.22$, which aligns with the observed effect on 2WikiMultiHopQA ($d = 0.235$ vs. Loop; achieved power = 0.79). The non-significant comparisons on HotpotQA (vs. Loop: $d = 0.016$) and MuSiQue (vs. Loop: $d = 0.087$) reflect *genuinely small effects*: detecting these with 80% power would require $n > 43{,}000$ and $n > 1{,}400$ respectively. This is consistent with the complexity-dependent pattern discussed in §7.1: the agentic advantage is substantial on

Table 5: Cross-model validation (RQ1, gpt-5-mini). $n = 200$ per dataset (FinanceBench: $n = 150$). LLM-as-Judge: gpt-4.1-nano (same independent judge as Gemini experiments). Best per dataset in **bold**.

	HotpotQA			**2WikiMultiHopQA**			**MuSiQue**			**FinanceBench**		
Pipeline	EM	F1	Judge	EM	F1	Judge	EM	F1	Judge	EM	F1	Judge
Naive RAG	.485	.657	.755	.335	.400	.405	.310	.441	.425	.140	.321	**.540**
CRAG Replica	**.535**	**.698**	.780	**.550**	**.637**	**.655**	.370	.491	.460	**.147**	**.343**	.520
Single-Pass	.505	.678	.735	.390	.465	.485	**.375**	.504	.490	.140	.308	.480
Loop Refinement	.510	.689	.750	.425	.510	.540	.370	.505	.485	.160	.328	.507
TARA	.510	.697	**.790**	.485	.551	.560	**.375**	**.516**	**.505**	.133	.317	.513
Δ *vs. CRAG*	−.025	−.001	+.010	−.065	−.086	−.095	+.005	+.025	+.045	−.014	−.026	−.007

complex multi-hop questions and diminishes on simpler tasks where adaptive retrieval strategies provide little additional benefit.

5.1.2. Cross-Model Validation

To assess whether the observed patterns are model-dependent, we replicate the RQ1 experiment using gpt-5-mini[3], a reasoning-oriented model from a different provider and architecture family. Table 5 presents the results.

Model–pipeline interaction effect. The cross-model experiment reveals a striking reversal: with gpt-5-mini, CRAG Replica outperforms TARA on 2WikiMultiHopQA (F1 0.637 vs. 0.551, $p = 0.004$) and FinanceBench (F1 0.343 vs. 0.317, $p = 0.042$). This contrasts sharply with the Gemini results, where TARA leads on both datasets. The reversal is driven by what we term *Reasoning Model Refusal Asymmetry*: the reasoning capabilities of gpt-5-mini lead it to apply higher evidentiary standards within the ReAct framework, resulting in a 30% answer refusal rate on 2WikiMultiHopQA (compared to 18% for CRAG). CRAG's rigid judge–correct–generate pipeline precludes refusal, ensuring an answer is always produced. We analyze this phenomenon in detail in §7.3.

[3]Model ID: `gpt-5-mini`, with `reasoning_effort=low`.

Table 6: Average latency per question (seconds, Gemini Flash Lite, $\tau = 40$). Latency is dominated by LLM API calls; the agent averages 5–6 tool invocations per question.

Pipeline	**HotpotQA**	**2Wiki**	**MuSiQue**	**Finance**
Naive RAG	0.5	0.3	0.3	0.3
CRAG Replica	0.4	0.1	0.1	0.1
Single-Pass	4.2	3.0	3.5	4.9
Loop Refinement	3.1	1.7	3.6	2.7
TARA	14.9	13.6	15.2	16.8

Consistent findings. Despite the model–pipeline interaction, several patterns remain consistent across both models: (1) TARA leads on MuSiQue (F1 0.516, Judge 0.505), the hardest multi-hop dataset; (2) Naive RAG achieves the highest Judge score on FinanceBench with both models (0.580 for Gemini, 0.540 for gpt-5-mini); (3) HotpotQA shows compressed performance ranges with both models. The LLM-as-Judge also resolves an apparent tie on HotpotQA: TARA achieves 0.790 vs. CRAG's 0.780, suggesting subtle quality advantages that F1 cannot capture.

5.1.3. Latency Analysis

Table 6 reports average per-question latency for the Gemini configuration.

TARA requires 14–17 seconds per question, approximately 4–5× slower than Loop Refinement and 30–50× slower than Naive RAG. This overhead stems from the ReAct reasoning loop: each iteration involves LLM inference for thought generation, tool selection, and observation processing. With an average of 5.4–6.2 tool calls per question (see §5.2), the agent performs substantially more LLM calls than the fixed-loop approach. With gpt-5-mini, latency increases further to 45.7s per question on 2WikiMultiHopQA due to the model's reasoning overhead. CRAG requires 147.8s per question with gpt-5-mini, as its multi-stage pipeline involves 45.2 LLM calls per question. We discuss the latency–quality trade-off and practical implications in §7.

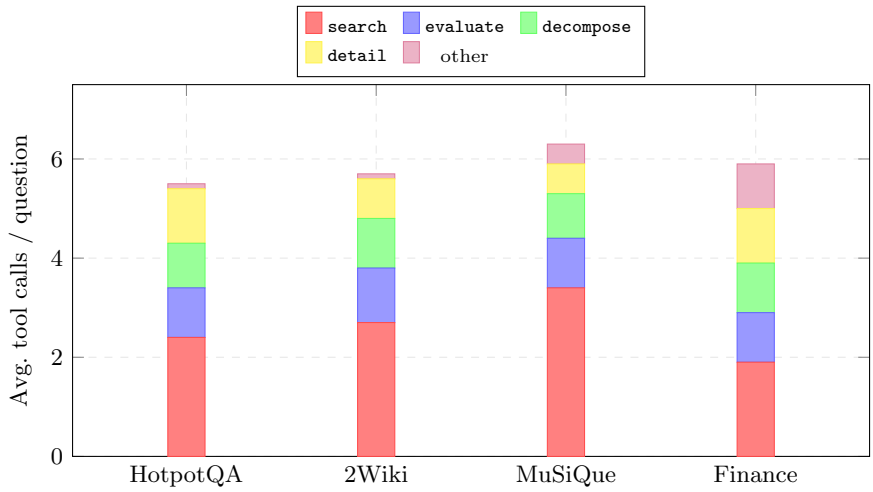

Figure 5: Tool call distribution per question (RQ2, $n = 200$). Search dominates across all datasets, with MuSiQue requiring the most tool calls (6.2/q) due to its 3–4 hop complexity. FinanceBench shows the most diverse tool usage, including domain-specific `calculate` and `list_sections` (grouped as "other").

5.2. RQ2: Tool Usage and Trajectory Analysis

How does the ReAct agent use its tools, and what behavioral patterns emerge?

5.2.1. Tool Frequency

Table 7 and Figure 5 report the average number of tool calls per question across all four datasets.

Search dominance. The agent calls `search_passages` 1.9–3.4 times per question, making it the most frequently used tool. MuSiQue exhibits the highest search frequency (3.4), consistent with its 3–4 hop structure requiring multiple independent evidence retrieval passes. Conversely, FinanceBench requires the fewest searches (1.9), reflecting its small corpus where initial retrieval often suffices.

Progressive information disclosure. `get_passage_detail` is called 0.6–1.1 times per question, indicating that the agent actively employs a two-stage retrieval

Table 7: Average tool calls per question ($n = 200$ per dataset). The agent prioritizes search and decomposition, with evaluation called at least once per question via mandatory fallback.

Tool	**HotpotQA**	**2Wiki**	**MuSiQue**	**Finance**
`search_passages`	2.4	2.7	3.4	1.9
`evaluate_passages`	1.0	1.1	1.0	1.0
`decompose_query`	0.9	1.0	0.9	1.0
`get_passage_detail`	1.1	0.8	0.6	1.1
`list_document_sections`	0.1	0.1	0.4	0.5
`calculate`	–	–	–	0.4
`get_terminology`	<0.1	0.0	<0.1	<0.1
Total tools/question	**5.4**	**5.7**	**6.2**	**5.8**

strategy: first scanning search previews (200-character summaries), then retrieving full text for promising passages. This progressive disclosure reduces context window consumption while maintaining retrieval precision.

Structure-aware navigation. `list_document_sections` is rarely used on Wikipedia-based datasets (≤ 0.1 per question) but more frequently on MuSiQue (0.4 per question) and FinanceBench (0.5 per question), where complex document structures benefit from hierarchical navigation. In contrast, `get_terminology` is virtually unused across all datasets (< 0.1 calls per question), indicating that the vocabulary gap between questions and corpus language is not a significant bottleneck for these benchmarks.

Domain-adaptive tool selection. On FinanceBench, the agent invokes `calculate` (0.4 calls per question) for numerical computations such as margin calculations and year-over-year comparisons. This tool is completely unused on Wikipedia-based datasets, demonstrating the agent's ability to adapt its tool repertoire to the domain without explicit configuration.

5.2.2. Trajectory Patterns

Analysis of ReAct trajectories reveals three dominant behavioral patterns:

1. **Decompose–Search–Evaluate** (most common): The agent decomposes the question into sub-questions, searches each independently, and evaluates the combined passage set. This is the canonical multi-hop retrieval strategy.
2. **Search–Evaluate** (simple cases): For questions identified as single-hop, the agent skips decomposition and directly searches with the preprocessed query before evaluating.
3. **Decompose–Search–Evaluate–Refine** (iterative refinement): When the initial evaluation score falls below the quality threshold, the agent uses evaluation feedback to refine the query, performs an additional search, and re-evaluates.

Notably, the agent *converges on* these structured protocols through ReAct reasoning, despite having access to a broader set of tools and no hard-coded execution sequence. The protocol is suggested in the agent's signature docstring but not enforced—the agent could deviate but consistently converges on these patterns across all 800+ questions evaluated.

5.3. RQ5: DSPy Optimization Effect

Does DSPy's declarative pipeline and automatic optimization improve answer quality compared to manual prompt engineering?

Table 8 compares four prompt engineering approaches. The test set sizes (N) differ from RQ1 because 50 examples per dataset are reserved for DSPy optimization training and 20 for validation.

Structural benefit: Manual → DSPy Unopt. Replacing hand-crafted JSON prompt templates with DSPy typed Signatures yields substantial improvements *without any optimization* on three datasets: F1 increases of +0.071 (HotpotQA), +0.126 (2WikiMultiHopQA), and +0.164 (FinanceBench). The FinanceBench

Table 8: DSPy optimization results (RQ5, Gemini Flash Lite). "Manual" uses hand-crafted JSON templates; "Unopt" uses DSPy typed Signatures without optimization; Bootstrap and MIPROv2 add automatic few-shot/instruction optimization. Best per dataset in **bold**. N: test set size after train/validation split.

	HotpotQA		**2WikiMultiHopQA**		**MuSiQue**		**FinanceBench**		
Variant	EM	F1	EM	F1	EM	F1	EM	F1	N
Manual Prompt	.408	.587	.308	.476	.338	.466	.013	.255	–
DSPy Unopt	.508	.658	.485	.602	.300	.396	.188	.419	–
DSPy + Bootstrap	**.508**	**.664**	**.554**	**.646**	**.469**	**.544**	**.200**	**.431**	130/80
DSPy + MIPROv2	.500	.655	.400	.528	.423	.503	.175	.409	130/80
Δ *Manual→Unopt*	+.100	+.071	+.177	+.126	−.038	−.070	+.175	+.164	
Δ *Manual→Best*	+.100	+.077	+.246	+.170	+.131	+.078	+.187	+.176	

improvement is particularly notable: from F1 0.255 to 0.419, with EM increasing from 0.013 to 0.188. MuSiQue is the exception, where Unopt substantially underperforms Manual (−0.070 F1), suggesting that DSPy's structured output constraints can conflict with question patterns that benefit from free-form generation. This dataset-dependent variance highlights that the structural benefit is not universal but depends on alignment between typed Signature constraints and task characteristics.

Optimization benefit: Unopt → Bootstrap/MIPROv2. Automatic optimization provides additional gains, but the effect is *dataset-dependent*:

- **BootstrapFewShot** is the most consistently effective optimizer, achieving the best F1 on all four datasets: HotpotQA (0.664, +0.006 over Unopt), 2WikiMultiHopQA (0.646, +0.044), MuSiQue (0.544, +0.148), and FinanceBench (0.431, +0.012). The large MuSiQue gain (+0.148) demonstrates that few-shot examples from successful execution traces can recover performance lost by the Unopt variant's rigid structure.
- **MIPROv2** provides strong gains on MuSiQue (0.503, +0.107 over Unopt) but underperforms Bootstrap on the remaining datasets. The jointly optimized instructions appear most beneficial for complex reasoning chains.

- **Optimization rescues structural mismatch**: On MuSiQue, where Unopt underperforms Manual, both optimizers substantially surpass Manual (Bootstrap +0.078, MIPROv2 +0.037 F1), demonstrating that automatic optimization can compensate for suboptimal Signature–task alignment.

Key finding. DSPy provides a two-level contribution: (1) the *structural benefit* of typed Signatures yields F1 improvements of +0.071–0.164 on three of four datasets, and (2) automatic optimization provides additional gains, with BootstrapFewShot achieving the best F1 on all four datasets. Critically, optimization can rescue structural mismatch: on MuSiQue, where Unopt underperforms Manual, Bootstrap still achieves the overall best F1 (+0.078 over Manual). The total improvement from Manual to best DSPy variant ranges from +0.077 to +0.176 F1, supporting the practical value of declarative LM programming for RAG systems.

6. Ablation Studies

We conduct two ablation studies to analyze the contribution of individual components within TARA: the evaluation tool configuration (RQ3, §6.1) and the structure-aware retrieval tools (RQ4, §6.2). Table 11 additionally provides a consolidated view of all ablation variants on a single metric (F1) for cross-component comparison.

Note on baseline values. Tables 9 and 10 report results from their respective experiment runs; the "Full system" baseline is shared across both. Table 11 consolidates values from these individual RQ tables for cross-component comparison. Minor differences between the Full system F1 here and Table 3 (typically ≤ 0.02) arise from non-determinism in LLM API calls at temperature = 0.

6.1. RQ3: Evaluation Tool Variants

Does 4-dimensional quality assessment as an agent tool improve decision-making compared to 1-dimensional or no evaluation?

We compare three variants of the evaluation tool within the TARA pipeline:

Table 9: Evaluation tool ablation (RQ3, $n = 200$). The w/o Eval variant consistently underperforms variants with evaluation, confirming the evaluation tool's role as a quality gate. 1D and 4D perform comparably, with 1D marginally ahead on 3/4 datasets. Best per dataset in **bold**.

	HotpotQA		**2WikiMultiHopQA**		**MuSiQue**		**FinanceBench**	
Variant	EM	F1	EM	F1	EM	F1	EM	F1
Agent + 4D	.525	.674	**.500**	**.611**	**.345**	.449	.180	.411
Agent + 1D	**.530**	**.680**	.505	.608	.340	**.455**	**.187**	**.421**
Agent w/o Eval	.525	.674	.475	.580	.340	.438	.153	.387
Δ *1D vs. w/o*	+.005	+.006	+.030	+.028	=	+.017	+.034	+.034
Δ *4D vs. w/o*	=	=	+.025	+.031	+.005	+.011	+.027	+.024

- **Agent + 4D**: Full 4-dimensional evaluation (Relevance, Coverage, Specificity, Sufficiency), with per-dimension scores and refinement feedback.
- **Agent + 1D**: Single holistic quality score (0–100) with refinement feedback but no dimensional breakdown.
- **Agent w/o Eval**: Evaluation tool removed entirely; the agent relies solely on its own reasoning about passage quality.

Evaluation as quality gate. At the $n = 200$ scale, a clear pattern emerges that was not discernible in pilot experiments: the **w/o Eval variant consistently ranks last** across all four datasets (F1: 0.674, 0.580, 0.438, 0.387). The quality gap is most pronounced on datasets where retrieval is challenging: 2WikiMultiHopQA ($\Delta = +0.028$–0.031 for 1D/4D over w/o Eval) and FinanceBench ($\Delta = +0.024$–0.034). On HotpotQA, where retrieval is relatively easy, the evaluation tool provides negligible benefit ($\Delta \leq 0.006$).

This confirms that the evaluation tool functions as a *quality gate mechanism*: by providing an explicit quality checkpoint, it prevents the agent from terminating prematurely with insufficient evidence. Without the gate, the agent must rely entirely on its own implicit quality assessment, which proves less reliable on difficult retrieval tasks.

1D vs. 4D: granularity is secondary. The 1D holistic score marginally outperforms the 4D dimensional breakdown on 3/4 datasets (F1 differences ≤ 0.010). This suggests that the *presence* of evaluation matters more than its *granularity*—a single quality score with feedback is sufficient for the agent to make informed refinement decisions. The 4D variant retains value as a diagnostic tool, providing interpretable dimension-level feedback (*e.g.*, "Coverage is low—consider searching for the second entity") that can aid debugging and analysis, even when it does not improve aggregate scores.

6.2. RQ4: Structure-Aware Retrieval Tools

Do structure-aware retrieval tools (section index, terminology mapping) improve agent performance?

We evaluate four configurations across all datasets to assess whether document structure navigation tools provide benefits beyond standard keyword search:

- **Full Tools**: All tools including `list_document_sections` and `get_terminology`.
- **w/o Section**: Section browsing tool removed.
- **w/o Term**: Terminology mapping tool removed.
- **w/o Both**: Both structure-aware tools removed (core tools only).

Dataset-dependent benefit. Table 10 reveals that the value of structure-aware tools is *dataset-dependent*:

- **MuSiQue and FinanceBench**: Full Tools achieves the highest F1 on both datasets, with consistent +0.027 F1 improvement over w/o Both. On MuSiQue, this reflects the benefit of hierarchical document navigation for 3–4 hop questions that require traversing multiple document sections. On FinanceBench, section browsing helps navigate the structured hierarchy of SEC filings (10-K sections, financial statement tables).

Table 10: Structure-aware tool ablation (RQ4, $n = 200$). Structure tools provide benefits on complex, multi-hop datasets (MuSiQue, FinanceBench) but not on simpler Wikipedia-based datasets (HotpotQA, 2WikiMultiHopQA). Best per dataset in **bold**.

	HotpotQA		**2WikiMultiHopQA**		**MuSiQue**		**FinanceBench**	
Variant	EM	F1	EM	F1	EM	F1	EM	F1
Full Tools	**.525**	.674	.500	.611	**.345**	**.449**	**.180**	**.411**
w/o Section	.495	.656	**.505**	**.621**	.335	.445	.153	.390
w/o Term	.500	.654	.480	.593	.335	.433	.153	.388
w/o Both	**.525**	**.677**	.485	.597	.320	.422	.153	.384
Δ *Full vs. w/o Both*	=	−.003	+.015	+.014	+.025	+.027	+.027	+.027

- **HotpotQA**: Removing both tools marginally *improves* F1 (0.677 vs. 0.674, $\Delta = +0.003$). Wikipedia articles used in HotpotQA have simple, flat structures where section navigation provides no advantage over keyword-based search.
- **2WikiMultiHopQA**: The w/o Section variant achieves the highest F1 (0.621 vs. 0.611 for Full), a mixed result. We attribute this to the action space effect: additional tools increase the agent's decision complexity, and on datasets where these tools provide marginal retrieval value, the expanded action space can slightly degrade performance.

Terminology tool. The `get_terminology` tool shows minimal impact across all datasets: removing it (w/o Term vs. Full) yields small, inconsistent changes ($|\Delta| \leq 0.020$). This is consistent with the tool frequency analysis (Table 7), where `get_terminology` is called < 0.1 times per question. The vocabulary gap between user questions and corpus language is not a primary bottleneck for these benchmarks, though the tool may prove more valuable in highly specialized domains with extensive jargon (*e.g.*, legal, medical).

Extensibility framework. While the aggregate benefit of structure tools is modest, the *architecture* that enables domain-specific tool integration is a meaningful contribution. The agent's ability to selectively invoke structure tools on

Table 11: Consolidated ablation results (F1, Gemini Flash Lite, $n = 200$). Each variant modifies one component of the full TARA pipeline. Best per dataset in **bold**.

Component	Variant	HotpotQA	2Wiki	MuSiQue	FinanceBench
Full system	Agent + All Tools	.674	.611	.449	.411
Evaluation (RQ3)	1D Evaluation	**.680**	.608	**.455**	**.421**
	w/o Evaluate Tool	.674	.580	.438	.387
Retrieval (RQ2)	w/o Search (no re-retrieval)	.621	.541	.420	.393
Structure (RQ4)	w/o Section Index	.656	**.621**	.445	.390
	w/o Term Index	.654	.593	.433	.388
Agent (RQ1)	w/o Agent (for-loop)	.636	.495	.399	.400
Prompt (RQ5)	Manual Prompt	.614	.488	.461	.258

datasets where they help (MuSiQue, FinanceBench) and ignore them where they do not (HotpotQA) demonstrates the viability of an extensible tool framework. Adding new domain-specific tools (*e.g.*, table parsers for financial data, citation resolvers for scientific documents) requires only implementing the tool function and registering it with the agent—no pipeline modification is needed.

6.3. Consolidated Ablation Summary

Table 11 provides a consolidated view of all ablation variants on F1.

Component importance ranking. Three observations emerge from the consolidated view:

1. **Re-retrieval is the most critical tool**: removing search (the agent's ability to re-retrieve) causes the largest consistent drop (F1 -0.018 to -0.070 vs. Full), confirming that iterative search is the agent's primary mechanism for quality improvement.
2. **Agentic reasoning outperforms fixed loops**: the for-loop variant consistently underperforms the full agent on multi-hop datasets (2Wiki: -0.116, MuSiQue: -0.050), demonstrating the value of autonomous tool selection over predetermined iteration. FinanceBench is the sole exception

(Loop 0.400 vs. Agent 0.411), consistent with the retrieval space saturation effect.

3. **MuSiQue Manual prompt anomaly**: the Manual prompt achieves 0.461 on MuSiQue, marginally exceeding the Full system (0.449) and the DSPy Unoptimized variant (0.396 in RQ5). This pattern—where DSPy's typed Signature constraints underperform free-form manual prompts—is consistent across both the ablation and RQ5 experiments for MuSiQue, suggesting a structural mismatch between Signature constraints and MuSiQue's question patterns. However, the gap is small (+0.012) and Manual collapses on FinanceBench (0.258, lowest overall), highlighting its fragility across domains.

7. Discussion

7.1. Complexity-Dependent Improvement

The most consistent finding across our experiments is that the agentic advantage is *proportional to question complexity*. On simple 2-hop questions (HotpotQA), TARA performs comparably to single-pass baselines ($\Delta \leq +0.022$ F1, not significant). On complex 2–4 hop questions (2WikiMultiHopQA), the improvement is substantial (+0.089 F1, $p < 0.001$) and consistent across all metrics. MuSiQue shows a positive but non-significant trend (+0.039 F1), positioning it between the simple and complex extremes.

This gradient has a natural explanation: complex multi-hop questions benefit from the agent's ability to *adaptively compose retrieval strategies*—decomposing questions, searching sub-questions independently, and evaluating the combined evidence. Simple questions that can be answered from a single well-retrieved passage set do not require this adaptive strategy, and the additional reasoning overhead provides no measurable benefit.

This suggests a practical deployment guideline: TARA should be selectively applied to questions identified as complex or multi-hop, while simpler questions can be routed to faster single-pass pipelines. This aligns with Adaptive-

RAG's [33] approach of routing by question complexity, though our system handles the complex-question case more effectively through tool-augmented reasoning.

7.2. Boundary Conditions: Retrieval Space Saturation

Results on FinanceBench reveal a consistent boundary condition for agentic refinement: TARA underperforms Loop Refinement on both Gemini (-0.014 F1) and gpt-5-mini (-0.011 F1), with Naive RAG achieving the highest LLM-as-Judge scores on both models (0.580 and 0.540, respectively). Statistical testing confirms that no pairwise comparison on FinanceBench reaches significance, indicating a genuine ceiling rather than sampling noise.

We identify three contributing factors:

Small retrieval space. FinanceBench contains only 211 passages, compared to 5,000–66,000 for the multi-hop datasets. In such a constrained retrieval space, the initial hybrid retrieval already surfaces most relevant passages. The agent's iterative search provides diminishing returns because there are few additional passages to discover—a phenomenon we term *retrieval space saturation.* This is supported by the compressed performance range: the top four pipelines span only 0.019 F1 (0.381–0.400) on Gemini.

Answer distortion through over-refinement. The LLM-as-Judge results reveal that Naive RAG produces the best-judged answers on FinanceBench. This suggests that iterative refinement—whether agentic or loop-based—can *distort* answers on domain-specific questions. Financial questions often require precise numerical answers or specific terminology; additional refinement cycles may introduce hedging language, rounding errors, or over-elaboration that reduce answer precision. We observe that agentic answers average 33.9 words per response compared to 12.7 for reference answers, indicating verbosity that penalizes exact match scoring.

Evaluation score miscalibration. The evaluation tool produces higher scores on FinanceBench questions that ultimately receive lower F1 scores. Questions with evaluation scores ≥ 80 achieve average F1 of only 0.289, while questions where evaluation errors occur achieve higher F1 (0.452). This suggests that the evaluation tool is not well-calibrated for domain-specific retrieval quality, potentially triggering unnecessary refinement cycles when the initial retrieval is already adequate.

These findings imply that agentic refinement is most valuable when the retrieval space is large enough to benefit from iterative exploration and when answer precision is compatible with multiple retrieval passes. Enterprise deployments on small, well-curated corpora may benefit more from simpler pipeline configurations.

7.3. Model–Pipeline Interaction Effect

The cross-model experiment reveals a previously uncharacterized interaction between model architecture and pipeline design. Over-refusal—the tendency of safety-aligned LLMs to reject innocuous queries—is a well-documented phenomenon [75, 76], and recent work shows that reasoning fine-tuning degrades abstention calibration overall, making models less reliable at distinguishing answerable from unanswerable questions [77]. We observe a manifestation of this miscalibration in the interaction between reasoning models and agentic pipelines. With Gemini Flash Lite (a generation-focused model), TARA consistently outperforms CRAG on multi-hop datasets. With gpt-5-mini (a reasoning-focused model), the pattern *reverses*: CRAG outperforms TARA on 2WikiMultiHopQA (+0.086 F1, $p = 0.004$).

Reasoning Model Refusal Asymmetry. We trace this reversal to differential refusal behavior across pipeline architectures. Table 12 quantifies the answer refusal rates (responses containing "Cannot determine," "insufficient information," or equivalent).

The mechanism operates as follows:

Table 12: Answer refusal rates (%) on 2WikiMultiHopQA by model and pipeline. Reasoning models (gpt-5-mini) refuse more frequently in agentic pipelines, while generation models (Gemini) show the opposite pattern.

	gpt-5-mini		**Gemini**	
Pipeline	**TARA**	**CRAG**	**TARA**	**CRAG**
Refusal rate	30%	18%	14%	34%
F1 (all)	.551	.637	.584	.255
F1 (non-refused)	.712	–	–	–

- **gpt-5-mini in ReAct**: The model's reasoning capabilities lead it to apply stringent evidentiary standards. Even when the evaluation tool reports adequate quality (mean score 66, above the threshold of 40), the agent independently judges the evidence insufficient and produces a refusal response. Each refusal scores F1 = 0.0, dragging the average down. When the agent *does* answer, its F1 is 0.712—substantially higher than CRAG's overall 0.637—indicating that refusals, not answer quality, drive the performance gap.
- **CRAG's forced-answer pipeline**: CRAG's rigid judge–correct–generate architecture does not permit the model to refuse. The pipeline always produces an answer, converting some refusal-prone questions into correct (or partially correct) responses.
- **Gemini in ReAct**: As a generation-focused model, Gemini is less conservative in its evidentiary standards. It refuses only 14% of the time in the agentic pipeline, allowing TARA to outperform CRAG (which, with Gemini, refuses 34% due to CRAG's stricter binary evaluation triggering more "incorrect" classifications).

Qualitative evidence. Trajectory analysis of divergent cases illustrates the mechanism concretely. On 2WikiMultiHopQA, we identify 19 questions where gpt-5-mini refuses but Gemini answers—and Gemini is correct on 16 of these (84%).

In a representative case, both models identify the relevant directors and retrieve birth date information for one; the evaluation tool assigns scores of 74 (gpt-5-mini) and 100 (Gemini), both above the quality threshold of 40. Gemini answers correctly, but gpt-5-mini responds: "*Cannot determine. Monte Hellman was born in 1932; [the other director's] birth year is not provided.*" The reasoning model applies a stricter evidentiary standard—requiring *both* birth dates to be explicitly stated—while the generation model infers the answer from available evidence. This pattern, where the model overrides the evaluation tool's "output" recommendation based on its own implicit quality judgment, recurs across the divergent cases. In 3 of 19 cases, Gemini's answer was incorrect, suggesting that the stricter standard occasionally prevents errors—but at a disproportionate cost, as the remaining 16 cases represent lost correct answers. While the sample is small ($n = 19$), the consistency of the mechanism across cases parallels the miscalibrated abstention behavior documented in broader LLM evaluations [77].

Implications. This finding has two practical implications. First, pipeline selection should consider the base model's reasoning characteristics: reasoning-oriented models may benefit from forced-answer pipelines or calibrated refusal thresholds, while generation-oriented models are better suited to agentic frameworks. Second, the refusal asymmetry suggests that evaluation tools should be designed to provide calibrated confidence signals that interact predictably with different model types, rather than relying on the model's implicit evidentiary standards.

Efficiency perspective. Despite CRAG's higher F1 with gpt-5-mini on 2WikiMultiHopQA, it requires 45.2 LLM calls per question (147.8 seconds) compared to 9.9 calls (45.7 seconds) for TARA—a 4.6× cost premium. Combined with Gemini Flash Lite, TARA achieves the optimal cost–performance balance across our experimental configurations.

7.4. Latency–Quality Trade-off

TARA incurs 4–5× latency overhead compared to Loop Refinement (14–17s vs. 2–4s per question on Gemini). This trade-off must be evaluated in context:

- **Absolute latency.** 15 seconds per question is acceptable for many enterprise knowledge management scenarios where answer quality takes precedence over response time (*e.g.*, compliance checking, due diligence, research assistance).
- **Cost attribution.** The latency is dominated by LLM API calls (5–6 tool invocations per question, each requiring LLM inference). With faster or locally deployed models, the absolute latency would decrease proportionally.
- **Quality payoff.** The F1 improvement of +0.089 on 2WikiMultiHopQA ($p < 0.001$) represents a statistically significant quality gain that justifies the latency cost in accuracy-critical applications.
- **Selective deployment.** As discussed in §7.1, the agentic approach should be selectively applied to complex questions. A two-tier system that routes simple questions to Single-Pass (3–5s) and complex questions to TARA (14–17s) would achieve an effective average latency well below 15s while capturing the quality benefits.

7.5. DSPy: Structure vs. Optimization

The RQ5 results reveal an important decomposition of DSPy's contribution:

The *structural benefit*—replacing manual JSON prompts with typed Signatures—yields F1 improvements of +0.071–0.164 on three of four datasets without any optimization. This benefit is essentially "free": it requires no training data, no optimization compute, and no hyperparameter tuning. However, the structural benefit is not universal: on MuSiQue, Unopt substantially underperforms Manual (−0.070 F1), indicating that typed Signature constraints can conflict with question patterns that benefit from free-form generation.

The *optimization benefit*—BootstrapFewShot and MIPROv2—compensates for this limitation. BootstrapFewShot is the most consistently effective optimizer, achieving best F1 on all four datasets. Critically, on MuSiQue where Unopt underperforms Manual, Bootstrap still achieves the overall best F1 (+0.078 over Manual), demonstrating that automatic optimization can rescue structural mismatch between Signatures and task characteristics.

This two-level decomposition has practical implications: teams adopting DSPy can expect immediate gains from Signatures on most tasks, and when structural mismatch occurs, automatic optimization provides a reliable recovery mechanism.

Complementary contributions. A natural question is whether the agentic reasoning (C1) and DSPy integration (C4) produce synergistic effects or provide independent value. To test this, we conduct a 2×2 factorial analysis crossing pipeline type (Agentic vs. Loop) with prompt strategy (DSPy vs. Manual), computing the interaction term: Synergy $= F1_{\text{Agent+DSPy}} - F1_{\text{Agent+Manual}} - F1_{\text{Loop+DSPy}} + F1_{\text{Loop+Manual}}$. The interaction is near zero across all datasets (+0.015, +0.064, −0.019, −0.037 for HotpotQA, 2WikiMultiHopQA, MuSiQue, and FinanceBench respectively; all $|\text{interaction}| < 0.065$), indicating *no meaningful synergistic or antagonistic interaction.* The DSPy structural benefit is present in both pipelines: +0.095–0.110 F1 for Agentic, +0.011–0.099 for Loop (excluding the MuSiQue anomaly where Manual outperforms DSPy Unopt). This independence is expected because the two contributions operate at different abstraction layers: DSPy optimizes *prompt-level* representations (Signature types, few-shot examples), while agentic reasoning optimizes *runtime execution paths* (tool selection, iteration decisions). Practically, this means each contribution can be adopted independently, allowing teams to incrementally introduce agentic reasoning or DSPy optimization based on their deployment constraints.

7.6. Instruction-Guided Protocol Convergence

A noteworthy observation is that the ReAct agent consistently converges on structured retrieval protocols. Despite having access to six tools (four core, two

domain-adaptive) with no enforced execution order, the agent converges on a decompose–search–evaluate pattern across all 800+ questions in our evaluation (RQ2, §5.2).

This convergence is partly *instruction-guided*: the agent's Signature docstring recommends a protocol outline, providing an inductive bias toward structured behavior. However, the agent retains meaningful autonomy in execution: the specific tool ordering, call frequency (2–10 tools per question), and termination decisions vary substantially across questions and are not prescribed by the protocol. More notably, two behaviors arise without any instruction: the progressive information disclosure pattern (search previews followed by selective full-text retrieval) and domain-adaptive tool selection (`calculate` on FinanceBench, `list_document_sections` on complex documents). These suggest that the combination of typed Signatures, evaluation feedback, and ReAct reasoning creates sufficient structure for adaptive behavior without requiring hard-coded orchestration logic.

7.7. Limitations and Future Work

We acknowledge several limitations:

1. **Retrieval corpus.** We use each dataset's provided context paragraphs rather than a large-scale open-domain corpus (*e.g.*, DPR's Wikipedia dump [18]). This ensures controlled evaluation of the refinement mechanism, but limits direct comparison with methods evaluated on open-domain retrieval: for instance, IRCoT [39] reports F1 $\approx$ 0.61 on HotpotQA using full Wikipedia, while our closed-corpus setting yields higher absolute scores (0.658) partly because gold passages are guaranteed to be present. The retrieval space saturation observed on FinanceBench (§7.2) may not generalize to larger enterprise corpora.
2. **Model–pipeline interaction.** The Reasoning Model Refusal Asymmetry (§7.3), consistent with broader findings on LLM over-refusal [75] and degraded abstention calibration in reasoning models [77], demonstrates that pipeline effectiveness depends on the base model's characteristics.

Our cross-model validation covers two model families (Gemini Flash Lite and gpt-5-mini); we cannot isolate whether the observed refusal asymmetry stems from reasoning-specific fine-tuning or other model-family differences. Broader validation across open-source models (*e.g.*, LLaMA [4], DeepSeek-R1) and model sizes would strengthen generalizability.

3. **Latency.** The 4–5× latency overhead limits applicability to latency-sensitive scenarios. Techniques such as parallel tool execution, speculative decoding, and agent module distillation could mitigate this but are not explored.
4. **Structure-aware tools.** The dataset-dependent benefit of structure tools (RQ4) is limited to our evaluation datasets. Documents with richer hierarchical structure (*e.g.*, legal contracts, technical manuals, multi-chapter reports) may show stronger benefits, which we leave to future investigation.
5. **Evaluation calibration.** The evaluation tool's miscalibration on FinanceBench suggests that domain-specific calibration or learned evaluation thresholds may be necessary for optimal performance across diverse domains.
6. **Statistical power.** A sensitivity analysis shows that our $n = 200$ design can detect effects of $d \geq 0.22$ at 80% power (Bonferroni-corrected $\alpha = 0.0125$), which is sufficient for the primary 2WikiMultiHopQA result ($d = 0.235$). The non-significant HotpotQA ($d = 0.016$) and MuSiQue ($d = 0.087$) comparisons represent genuinely small effects that would require $n > 43{,}000$ and $n > 1{,}400$ to detect, consistent with the complexity-dependent improvement pattern. Our sample size follows the evaluation scale of comparable RAG studies [9, 10], though larger-scale evaluation could further refine effect size estimates.

8. Conclusion

We presented TARA, a framework that replaces fixed-loop self-corrective RAG with a ReAct-based agent equipped with six specialized tools for autonomous retrieval refinement. Implemented as a DSPy declarative pipeline, the framework enables both structured prompting through typed Signatures and automatic optimization through BootstrapFewShot and MIPROv2.

Our experiments across four datasets ($n = 200$) with bootstrap significance testing and cross-model validation demonstrated three principal findings. First, agentic refinement provides *complexity-dependent* improvements: statistically significant gains on complex multi-hop questions (F1 +0.089 on 2WikiMultiHopQA, $p < 0.001$) with diminishing benefits on simpler tasks and a boundary condition on small corpora (FinanceBench). Second, DSPy's typed Signatures improve F1 by 0.071–0.164 over manual prompt engineering on three of four datasets, with BootstrapFewShot achieving the best F1 on all four datasets and recovering performance even where Signatures alone underperform manual prompts. Third, cross-model validation reveals a *model–pipeline interaction effect*: reasoning-oriented models (gpt-5-mini) exhibit higher refusal rates in agentic pipelines, suggesting that pipeline selection should account for the base model's reasoning characteristics.

Trajectory analysis reveals consistent convergence on structured protocols (decompose–search–evaluate), guided by typed Signatures and ReAct reasoning, with domain-adaptive tool selection arising without explicit instruction. Ablation studies confirm that the evaluation tool serves as a quality gate mechanism, with 1D holistic scores proving sufficient. Structure-aware tools provide dataset-dependent benefits on complex documents, and re-retrieval emerges as the agent's most critical capability.

Future Work. Several directions merit investigation: (1) evaluating structure-aware tools on documents with rich hierarchical structure (*e.g.*, legal contracts, technical manuals); (2) reducing agent latency through parallel tool execution, speculative decoding, or agent module distillation; (3) addressing the Reason-

ing Model Refusal Asymmetry through calibrated refusal thresholds or forced-answer fallback mechanisms; and (4) extending the framework with additional domain-specific tools for tasks such as table parsing, cross-document reasoning, and temporal analysis.

CRediT authorship contribution statement

Ruo Lee: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All multi-hop QA datasets (HotpotQA, 2WikiMultiHopQA, MuSiQue) are publicly available. FinanceBench is available under CC-BY-NC-4.0. Code and experiment configurations are available at `https://github.com/comsa33/self-corrective-rag`.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used Claude (Anthropic) for code development assistance and manuscript drafting support. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, Vol. 30, 2017, pp. 5998–6008.

[2] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 27730–27744.

[3] OpenAI, GPT-4 technical report, arXiv preprint arXiv:2303.08774 (2023).

[4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971 (2023).

[5] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, ACM Transactions on Information Systems 43 (2) (2025) 1–55. doi:10.1145/3703155.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, Advances in Neural Information Processing Systems 33 (2020) 9459–9474.

[7] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv preprint arXiv:2312.10997 (2024).

[8] W. Yang, L. Some, M. Bain, B. Kang, A comprehensive survey on integrating large language models with knowledge-based methods, Knowledge-Based Systems (2025). doi:10.1016/j.knosys.2025.113503.

[9] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, arXiv preprint arXiv:2401.15884 (2024).

[10] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations (ICLR), 2024.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[12] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Mober, et al., DSPy: Compiling declarative language model calls into state-of-the-art pipelines, in: International Conference on Learning Representations (ICLR), 2024.

[13] K. Opsahl-Ong, M. J. Ryan, J. Hardy, J. Peng, O. Khattab, C. Liang, Optimizing instructions and demonstrations for multi-stage language model programs, arXiv preprint arXiv:2406.11695 (2024).

[14] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, C. D. Manning, HotpotQA: A dataset for diverse, explainable multi-hop question answering, in: Proceedings of EMNLP, 2018, pp. 2369–2380.

[15] X. Ho, A.-K. D. Nguyen, S. Sugawara, A. Aizawa, Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps, arXiv preprint arXiv:2011.01060 (2020).

[16] H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal, MuSiQue: Multihop questions via single hop question composition, Transactions of the Association for Computational Linguistics 10 (2022) 539–554.

[17] P. Islam, A. Ravi, P. Chandra, A. Munirsamy, S. Muthukrishnan, FinanceBench: A new benchmark for financial question answering, `https://github.com/patronus-ai/financebench` (2023).

[18] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781. doi:10.18653/v1/2020.emnlp-main.550.

[19] O. Khattab, M. Zaharia, ColBERT: Efficient and effective passage search via contextualized late interaction over BERT, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020, pp. 39–48. doi:10.1145/3397271.3401075.

[20] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2019, pp. 4171–4186. doi:10.18653/v1/N19-1423.

[21] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389.

[22] K. Guu, K. Lee, Z. Tung, P. Pasupat, M. Chang, REALM: Retrieval-augmented language model pre-training, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 3929–3938.

[23] G. Izacard, E. Grave, Leveraging passage retrieval with generative models for open domain question answering, in: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, 2021, pp. 874–880. doi:10.18653/v1/2021.eacl-main.74.

[24] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, E. Grave, Atlas: Few-shot learning with retrieval augmented language models, Journal of Machine Learning Research 24 (251) (2023) 1–43.

[25] X. Ma, Y. Gong, P. He, H. Zhao, N. Duan, Query rewriting in retrieval-augmented large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 5303–5315. doi:10.18653/v1/2023.emnlp-main.322.

[26] L. Gao, X. Ma, J. Lin, J. Callan, Precise zero-shot dense retrieval without relevance labels, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023, pp. 1762–1777. doi:10.18653/v1/2023.acl-long.99.

[27] Z. Peng, X. Wu, Q. Wang, Y. Fang, Soft prompt tuning for augmenting dense retrieval with large language models, Knowledge-Based Systems 309 (2025) 112758. doi:10.1016/j.knosys.2024.112758.

[28] L. Silva, L. Barbosa, Improving dense retrieval models with LLM augmented data for dataset search, Knowledge-Based Systems 297 (2024) 111740. doi:10.1016/j.knosys.2024.111740.

[29] G. V. Cormack, C. L. Clarke, S. Buettcher, Reciprocal rank fusion outperforms condorcet and individual rank learning methods, in: Proceedings of the 32nd International ACM SIGIR Conference, 2009, pp. 758–759.

[30] J. Zhao, Z. Zhuo, J. Zhang, Q. Ye, T. Yang, J. He, T. Chen, S. Chao, Meta-chunking: Learning efficient text segmentation via logical perception, in: International Conference on Learning Representations (ICLR), 2025.

[31] A. J. Jimeno Yepes, Y. You, J. Milczek, S. Laverde, R. Li, Financial report chunking for effective retrieval augmented generation, arXiv preprint arXiv:2402.05131 (2024).

[32] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (3) (2019) 535–547.

[33] S. Jeong, J. Baek, S. Cho, S. J. Hwang, J. C. Park, Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question

complexity, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2024, pp. 7036–7050. doi:10.18653/v1/2024.naacl-long.389.

[34] W. Xiong, X. L. Li, S. Iyer, J. Du, P. Lewis, W. Y. Wang, Y. Mehdad, W.-t. Yih, S. Riedel, D. Kiela, B. Oguz, Answering complex open-domain questions with multi-hop dense retrieval, in: International Conference on Learning Representations (ICLR), 2021.

[35] O. Khattab, C. Potts, M. Zaharia, Baleen: Robust multi-hop reasoning at scale via condensed retrieval, in: Advances in Neural Information Processing Systems, Vol. 34, 2021, pp. 27670–27682.

[36] C. Zhao, C. Xiong, J. Boyd-Graber, H. Daumé III, Multi-step reasoning over unstructured text with beam dense retrieval, in: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2021, pp. 4635–4641. doi:10.18653/v1/2021.naacl-main.368.

[37] J. Ferguson, M. Gardner, H. Hajishirzi, T. Khot, P. Dasigi, IIRC: A dataset of incomplete information reading comprehension questions, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 1137–1147. doi:10.18653/v1/2020.emnlp-main.86.

[38] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, G. Neubig, Active retrieval augmented generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 7969–7992. doi:10.18653/v1/2023.emnlp-main.495.

[39] H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal, Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions, in: Proceedings of the 61st Annual Meeting of the Association

for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 10014–10037. doi:10.18653/v1/2023.acl-long.557.

[40] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, M. Lewis, Measuring and narrowing the compositionality gap in language models, in: Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 5687–5711. doi:10.18653/v1/2023.findings-emnlp.378.

[41] R. Zhao, X. Li, S. Joty, C. Qin, L. Bing, Verify-and-edit: A knowledge-enhanced chain-of-thought framework, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 5823–5840. doi:10.18653/v1/2023.acl-long.320.

[42] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, W.-t. Yih, REPLUG: Retrieval-augmented black-box language models, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics, 2024, pp. 8371–8384. doi:10.18653/v1/2024.naacl-long.463.

[43] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlgay, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, M. Tenenholtz, MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning, arXiv preprint arXiv:2205.00445 (2022).

[44] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, arXiv preprint arXiv:2302.04761 (2023).

[45] B. Paranjape, S. Lundberg, S. Singh, H. Hajishirzi, L. Zettlemoyer, M. T. Ribeiro, ART: Automatic multi-step reasoning and tool-use for large language models, arXiv preprint arXiv:2303.09014 (2023).

[46] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with large language models, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[47] S. Hao, Y. Gu, H. Ma, J. Hong, Z. Wang, D. Wang, Z. Hu, Reasoning with language model is planning with world model, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 8154–8173.

[48] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[49] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, Y. Zhuang, HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[50] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, M. Sun, ToolLLM: Facilitating large language models to master 16000+ real-world APIs, in: International Conference on Learning Representations (ICLR), 2024.

[51] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, J. Tang, AgentBench: Evaluating LLMs as agents, in: International Conference on Learning Representations (ICLR), 2024.

[52] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, SWE-bench: Can language models resolve real-world GitHub issues?, in: International Conference on Learning Representations (ICLR), 2024.

[53] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, G. Neubig, WebArena: A realistic web envi-

ronment for building autonomous agents, in: International Conference on Learning Representations (ICLR), 2024.

[54] A. Singh, A. Ehtesham, S. Kumar, T. Talaei Khoei, Agentic retrieval-augmented generation: A survey on agentic RAG, arXiv preprint arXiv:2501.09136 (2025).

[55] X. Li, G. Jin, Y. Wang, D. Wang, W. Zhu, Z. Zeng, F. Luo, P. Ren, Z. Chen, Z. Ren, Search-o1: Agentic search-enhanced large reasoning models, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2025.

[56] M. Lee, S. Seo, E. Ahn, J. Jeon, D. Kim, PlanRAG: A plan-then-retrieval augmented generation for generative large language models as decision makers, arXiv preprint arXiv:2406.12430 (2024).

[57] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, A. Sabharwal, Decomposed prompting: A modular approach for solving complex tasks, in: International Conference on Learning Representations (ICLR), 2023.

[58] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, S. Singh, AutoPrompt: Eliciting knowledge from language models with automatically generated prompts, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 4222–4235.

[59] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, J. Ba, Large language models are human-level prompt engineers, in: International Conference on Learning Representations (ICLR), 2023.

[60] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, X. Chen, Large language models as optimizers, in: International Conference on Learning Representations (ICLR), 2024.

[61] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, C. Guestrin, J. Zou, Optimizing generative AI by backpropagating language model feedback, Nature 639 (8055) (2025) 609–616.

[62] O. Khattab, K. Santhanam, X. L. Li, D. Hall, P. Liang, C. Potts, M. Zaharia, Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP, arXiv preprint arXiv:2212.14024 (2023).

[63] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837.

[64] V. Mavi, A. Jangra, A. Jatowt, Multi-hop question answering, Foundations and Trends in Information Retrieval 17 (4) (2024) 457–586. doi:10.1561/1500000102.

[65] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T.-H. Huang, B. Routledge, W. Y. Wang, FinQA: A dataset of numerical reasoning over financial data, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 3697–3711. doi:10.18653/v1/2021.emnlp-main.300.

[66] V. Reddy, R. Koncel-Kedziorski, V. D. Lai, M. Krumdick, C. Lovering, C. Tanner, DocFinQA: A long-context financial reasoning dataset, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2024, pp. 445–458. doi:10.18653/v1/2024.acl-short.42.

[67] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-bench and chatbot arena, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[68] C.-Y. Lin, ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out, Association for Computational Linguistics, 2004, pp. 74–81.

[69] S. Es, J. James, L. Espinosa Anke, S. Schockaert, RAGAS: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 2024, pp. 150–158. doi:10.18653/v1/2024.eacl-demo.16.

[70] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.

[71] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, Y. Artzi, BERTscore: Evaluating text generation with BERT, in: International Conference on Learning Representations, 2020.

[72] Gemini Team, Google, Gemini: A family of highly capable multimodal models, arXiv preprint arXiv:2312.11805 (2023).

[73] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992. doi:10.18653/v1/D19-1410.

[74] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall/CRC, New York, 1993.

[75] J. Cui, W.-L. Chiang, I. Stoica, C.-J. Hsieh, OR-Bench: An over-refusal benchmark for large language models, in: International Conference on Machine Learning (ICML), 2025.

[76] Z. Zhang, W. Xu, F. Wu, C. K. Reddy, FalseReject: A resource for improving contextual safety and mitigating over-refusals in LLMs via structured reasoning, arXiv preprint arXiv:2505.08054 (2025).

[77] P. Kirichenko, M. Ibrahim, K. Chaudhuri, S. J. Bell, Abstention-Bench: Reasoning LLMs fail on unanswerable questions, arXiv preprint arXiv:2506.09038 (2025).